Chapter 6: Modeling the Event

Powerful research questions can be unpacked as soon as we have the tools for counting mentions of words over time. Previously, we've been unpacking research questions that stem from the tools we have – like the ability to follow particular speakers or words through time. But another approach, which leverages the power of historical analysis, is to start with the event and ask: what can computational approaches teach us about history that we previously did not know?

In the history of parliament, one of the most important events is the Reform Act of 1832, which granted the middle class the right to vote, and the Reform Act of 1867, which granted the right to vote to most working-class men living in cities.

To analyze how the reform acts changed parliament, we can compare the speakers and speeches a decade before and a decade after each event. Such an approach can be accomplished with the tools that have already been taught in previous chapters. However, it requires us to load three decades of data at a time.

0.0.1 Working With Big Data: Cautions Before Proceeding

In the previous chapters, we worked with intentionally small, subsetted datasets so the examples would run on a laptop. In this chapter, we take the opposite approach: we join multiple decades and create a much larger dataset first. We then filter it as needed. Doing so, we demonstrate another workflow for working with large-scale historical data on a computer with limited resources. This time, the analyst begin with the full corpus and progressively narrow the scope through filtering and subsetting.

Previously, we demonstrated techniques were we worked with chunks of the data at a time to save computer memory. We chunked the data so the code will be able to process it on a laptop.[1]

```r
# load some data (hansardr version needed)
# dates 1822-1842
library("hansardr")
library("tidyverse")
library("lubridate")

data("hansard_1820")
data("hansard_1830")
data("hansard_1840")

data("debate_metadata_1820")
data("debate_metadata_1830")
data("debate_metadata_1840")
data("speaker_metadata_1820")
data("speaker_metadata_1830")
```

```r
data("speaker_metadata_1840")

annotated_1820 <- hansard_1820 %>%
  left_join(debate_metadata_1820)  %>%
  mutate(year = year(speechdate)) %>%
  left_join(speaker_metadata_1820) %>%
  select(text, year, speechdate, suggested_speaker)

annotated_1830 <- hansard_1830 %>%
  left_join(debate_metadata_1830) %>%
  mutate(year = year(speechdate))  %>%
  left_join(speaker_metadata_1830) %>%
  select(text, year, speechdate, suggested_speaker)

annotated_1840 <- hansard_1840 %>%
  left_join(debate_metadata_1840) %>%
  mutate(year = year(speechdate))  %>%
  left_join(speaker_metadata_1840) %>%
  select(text, year, speechdate, suggested_speaker)

hansard <- bind_rows(annotated_1820, annotated_1830, annotated_1840)
```

`hansard` is a large dataset that includes three decades of parliamentary debates along with multiple layers of metadata that make the text analyzable. The debate metadata (which includes information such as the date of the speech and the title of the debate) and the speaker metadata (which includes the original speaker name and the disambiguated canonical name) are linked to each sentence through a shared `sentence_id`.

We can remove the variables that we are no longer using to save space ———

First, let's organize the data into two groupings – the words spoken prereform and those spoken postreform.

```r
rm(hansard, annotated_1820, annotated_1830, annotated_1840)
gc()
```

```
##            used  (Mb) gc trigger   (Mb)  max used   (Mb)
## Ncells  5759118 307.6   10813152  577.5   6140133  328.0
## Vcells 89333203 681.6  187684205 1432.0 178554850 1362.3
```

Let's break the data up into individual words and count how many words are in each category.

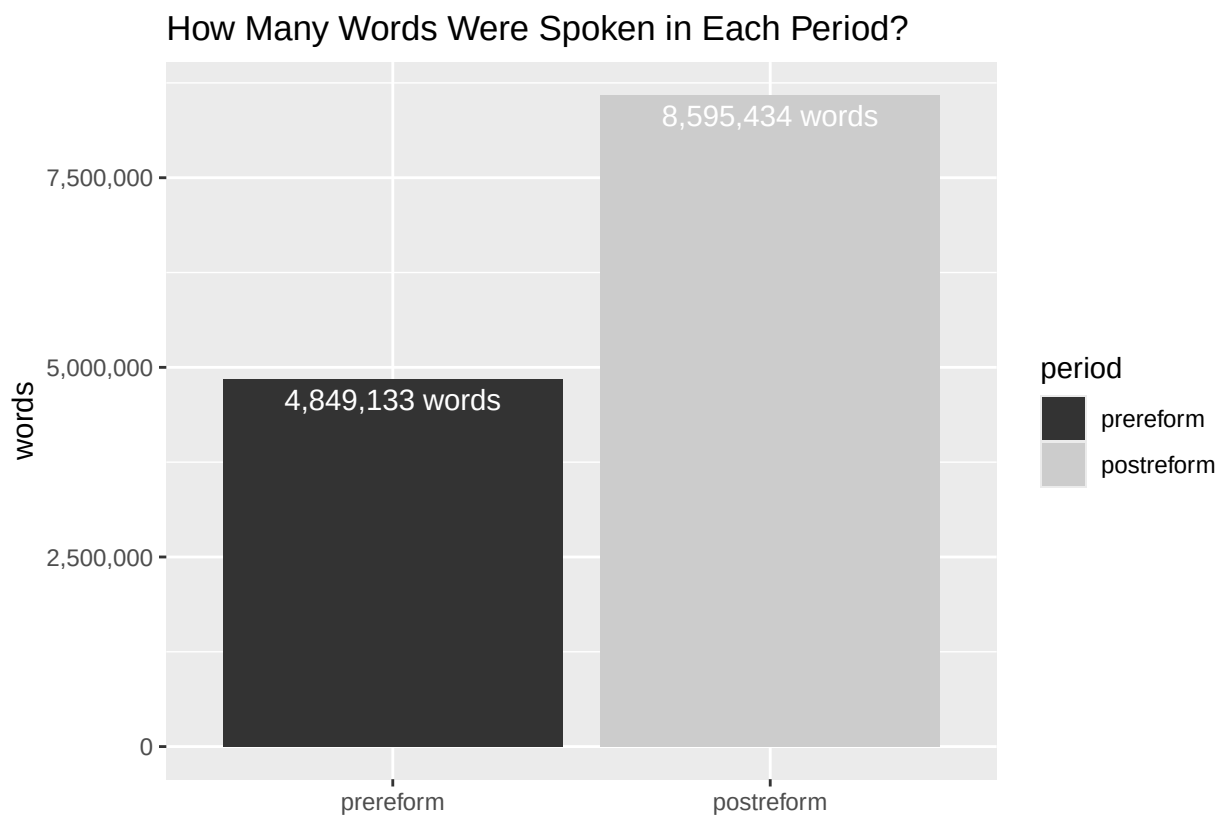
```
## Joining with `by = join_by(word)`
## Joining with `by = join_by(word)`
```

```r
# create a bar chart
library("scales") # a library useful for formatting numbers
```

```
##
## Attaching package: 'scales'
```

```
## The following object is masked from 'package:purrr':
##
##     discard

## The following object is masked from 'package:readr':
##
##     col_factor
```

```
ggplot(both_periods_word_count_totals, aes(x = period, y = total)) +
  geom_col((aes(fill = period))) +
  scale_fill_grey() +
  scale_y_continuous(label=comma) +
  geom_text(aes(label = (paste0(scales::comma(total), " words"))),
            colour = "white",
            vjust = 1.5) +
  labs(title = "How Many Words Were Spoken in Each Period?", x = "", y = "words")
```



Many more words were spoken in the ten years after the reform than the ten years before. How many more? Text mining means we never have to guess.

Table 6.1: The Top Ten Words Spoken in Prereform and Postreform Parliament

word	wordcount	period
house	59,045	prereform
hon	58,821	prereform
country	38,776	prereform
noble	34,335	prereform
bill	32,251	prereform
government	27,383	prereform
lord	25,402	prereform
time	21,347	prereform
question	20,757	prereform
subject	20,726	prereform
house	121,241	postreform
hon	119,156	postreform
noble	79,627	postreform
bill	66,044	postreform
lord	65,073	postreform
country	55,848	postreform
government	54,333	postreform
question	41,455	postreform
ireland	40,072	postreform
subject	38,767	postreform

```
## [1] 3746301
```

Next, let's look at the actual words spoken in each period and see how they compare.

```
##
## Attaching package: 'kableExtra'

## The following object is masked from 'package:dplyr':
##
##     group_rows
```

The analyst will discern that there are only very minor differences between the top words of both categories. What are we to make of that?

We should begin by admitting that these words tell us little that's new about parliament or history. The word "house" was frequently used in parliament to refer to the House of Commons and House of Lords, the two chambers of parliament. Members of the houses traditionally addressed each other as "honorable sir" (abbreviated "hon" in Hansard) and (when addressing peers of the realm) "noble sir," "noble sirs," or "my lord." The general matters of parliament concerned the state of the "country," the passing of "bills" into

law, the activities of the "government," various other "subjects," and the official matter of "questions" – a formal category of address about the business of parliament. To find the top words offers no real insight to any analyst who understands the basic business of parliament.

The only apparent difference in this table occurs in the words "time" – a top-ten word prereform but not postreform – and "Ireland" – a top-ten word postreform and prereform. If we looked at more than the top-ten words, even these differences would likely appear slight; the two words missing in this table are almost surely in the top twenty list for both periods – something we can easily check by adjusting the command `slice_head(n=10)` from 10 to 20.

In short, this step provides us with what data scientists refer to as a "validation" – that is, we have confirmed that the method of counting words tells us that the speech of parliament concerns the business of parliament.

A validation step is crucial to knowing that we are on the right track. If our attempt at counting (or any other statistical analysis) suggested that parliamentary speech in general concerned squids, whales, and cephalopods, or even obscure names of towns in Ireland, we would guess that something had gone wrong. Throughout the process of data analysis, we should use basic statistical measures as validation that the general idea is working. If our analysis tells us anything that doesn't make sense, we should rush to check our method – not revisit the laws of history.

0.0.2 Strategies for Reaching Insight

How do we move from validation to insight? There are many strategies. Let's think through several:

- 1) We can read a longer list of top words for each period, for instance the top 500 instead of the top 10. Keeping track of minute changes may tell us something. On the other hand, there will be a great many words to look at. If the differences are slight (like a #9 vs a #12 position for a certain word from one period to the next) we risk losing track of which words are important for which periods.
- 2) We can look at other aspects of the dataset, for instance the speaker column, asking how many words were spoken by each speaker in the prereform and postreform periods.
- 3) We can use a variety of statistical methods to aggregate information about the speakers that were the most different and the words whose expression was most different in the prereform and postreform periods.

In the sections that follow, we will undertake each of these steps in turn. They will give us different answers.

Step #1 is easily accomplished by a quick adjustment in code, tweaking the command `slice_head(n=10)` from 10 to 500. Please change the command for youself and inspect the results. Is it easy to interpret them?

Step #2 is easily accomplished by revisiting the code in greater depth. We won't need any new commands, but we will need to apply the functions we've already seen – `group_by()`, `summarize()`, and `n()` – to the `speaker` field rather than the `year` field.

Let's do that now to answer the question, how many words did each speaker speak in the prereform and postreform period?

0.1 Investigating Speakers' Wordcounts, Pre- and Post-Reform

```
## `summarise()` has grouped output by 'suggested_speaker'. You can override using
```

```
## the `.groups` argument.
## `summarise()` has grouped output by 'suggested_speaker'. You can override using
## the `.groups` argument.
```

```
## # A tibble: 6 x 4
##   suggested_speaker        year wordcount period
##   <chr>                   <dbl>     <int> <chr>
## 1 abraham_hume_1561        1826       702 prereform
## 2 adolphus_dalrymple_1968  1831       457 prereform
## 3 adolphus_dalrymple_1968  1832      1699 prereform
## 4 alexander_baring_1475    1830      5671 prereform
## 5 alexander_baring_1475    1831      7135 prereform
## 6 alexander_dawson_2455    1826      1935 prereform
```

```
rm(prereform_words, postreform_words)
```

Let's look at the speakers who spoke the most.

```
## # A tibble: 5 x 2
##   suggested_speaker    wordsperspeaker
##   <chr>                          <int>
## 1 robert_peel_1664             1982866
## 2 henry_brougham_1679          1616327
## 3 joseph_hume_1712             1336979
## 4 john_russell_1885            1046022
## 5 daniel_oconnell_2552          811831
```

0.2 Investigating Speaker Wordcount With Joins

With the data frame both_periods_speakers, we have wordcounts for each speaker for pre- and postreform. There's a great deal we can do with this data. First, let's perform some basic counts to help us to understand, in raw numbers, the basic features of prereform and postreform parliament.

We'll start by asking how many members of parliament stood for the ten years pre and post reform. We will label each with a "class" called "prereform only," "postreform only," or "continuous speakers." Note that these "classes" are fundamentally different than the "period" categorization we applied above, where we labeled speeches given before the 1832 reform law as "prereform" and those after the law as "postreform." Instead of labeling the years, we want to identify the speakers that spoke only in one period or the other, and those who spoke in both.

To identify these speakers, we will begin by returning to an earlier data frame – `prereform_speakers_wordcount` – and cleaning it to exclude those speakers who also appear in the dataset `postreform_speakers_wordcount`.

Information scientists theorize several ways to combine data through what is called "set theory." Two of the simplest joins are known as the "union" of two sets – what they have in common – and the "difference" between two sets – what is in one but not the other.

To find the prereform-only and postreform-only speakers, we will find the "difference" between the two sets by using a function named `anti_join()`. The `anti_join()` function excludes from one dataset all the data

that appear in a second dataset. If we begin with a dataset of all members of parliament who spoke prereform and perform an "anti-join" with the set of speakers who spoke postreform, the resulting dataset is a list of speakers who only spoke prereform.

```
## # A tibble: 6 x 2
##   suggested_speaker    class
##   <chr>                <chr>
## 1 george_shiffner_1709 prereform only
## 2 mr a ellis           prereform only
## 3 james_johnston_2695  prereform only
## 4 colonel wood         prereform only
## 5 mr curwen            prereform only
## 6 charles_abbot_1168   prereform only
```

How many speakers are there in our dataset? To answer the question, we just need to know the number of rows, given that the dataset lists one distinct speaker for each row. The function `nrow()` will provide an answer to how many rows there are in any dataset.

```
## [1] 1484
```

Performing the same operation again, beginning with postreform speeches and removing the rows of data that appear prereform, yields a dataset of speakers who only spoke after the reform.

```
## Joining with `by = join_by(suggested_speaker)`
```

```
## # A tibble: 6 x 2
##   suggested_speaker  class
##   <chr>              <chr>
## 1 aaron_chapman_3235 postreform only
## 2 aaron_chapman_3235 postreform only
## 3 aaron_chapman_3235 postreform only
## 4 aaron_chapman_3235 postreform only
## 5 aaron_chapman_3235 postreform only
## 6 aaron_chapman_3235 postreform only
```

How many speakers were there postreform?

```
## [1] 2438
```

We now have two sets of numbers – the count of unique speakers who spoke 1822-32 and 1832-42. The sets of numbers are new to history; there has never before been a count of these speakers. We can learn something from them. Before text mining, it was impossible to get a real impression of how many speakers were active in parliament before and after 1832. But the simple ability to count allows us to add detail to our accounts of the reform and its mechanisms.

The basic finding is that there were more speakers after the reform period. Our observation makes sense, because the reforms of 1832 introduced more popular consent into the democratic process. No longer

were members appointed in "pocket buroughs" by aristocrats who held the sole authority for electing the representative. Now, all seats were open to a popular vote (albeit of those rich enough to pay at least £10 per year in taxes – no mean sum). If we know that the number of seats in the House of Commons were set at 658 before and after the reform (which we learn by consulting a secondary source, XXXXXX), the numbers tell us that this reform introduced more variation and turnover in who occupied the seats of parliament.

Substantively more different members of parliament were elected in the ten years following 1832 than in the decade before. How many more members were there among the postreform only members than the prereform only members?

```
## [1] 954
```

We can also investigate speakers from a third category – those who were continuously in parliament, who held office before the reform and who were able to secure election after the reform as well. How many such speakers were there?

Please note that we do not have the data to ask which members of parliament who held office before 1832 were elected to office after 1832. We do not have the data to support that question. What we have in Hansard is only a record of who was recorded as having spoken. This is a rather more discrete query, because not every elected member actually speaks when they are in office, and some of those who spoke might have spoken so little that the clerks recording speeches took no notice of what they said.

Because recorded parliamentary speech is an index of importance, the members who were recorded as having spoken before and after 1832 offers a list of those members who were important enough to have taken notice of the public in both periods. The overlap of members who appear in both the pre- and post-reform records is a good proxy for which members mattered in the public imagination and met the criterion for having been elected both before and after the reform.Do we imagine this number to have been large or small?

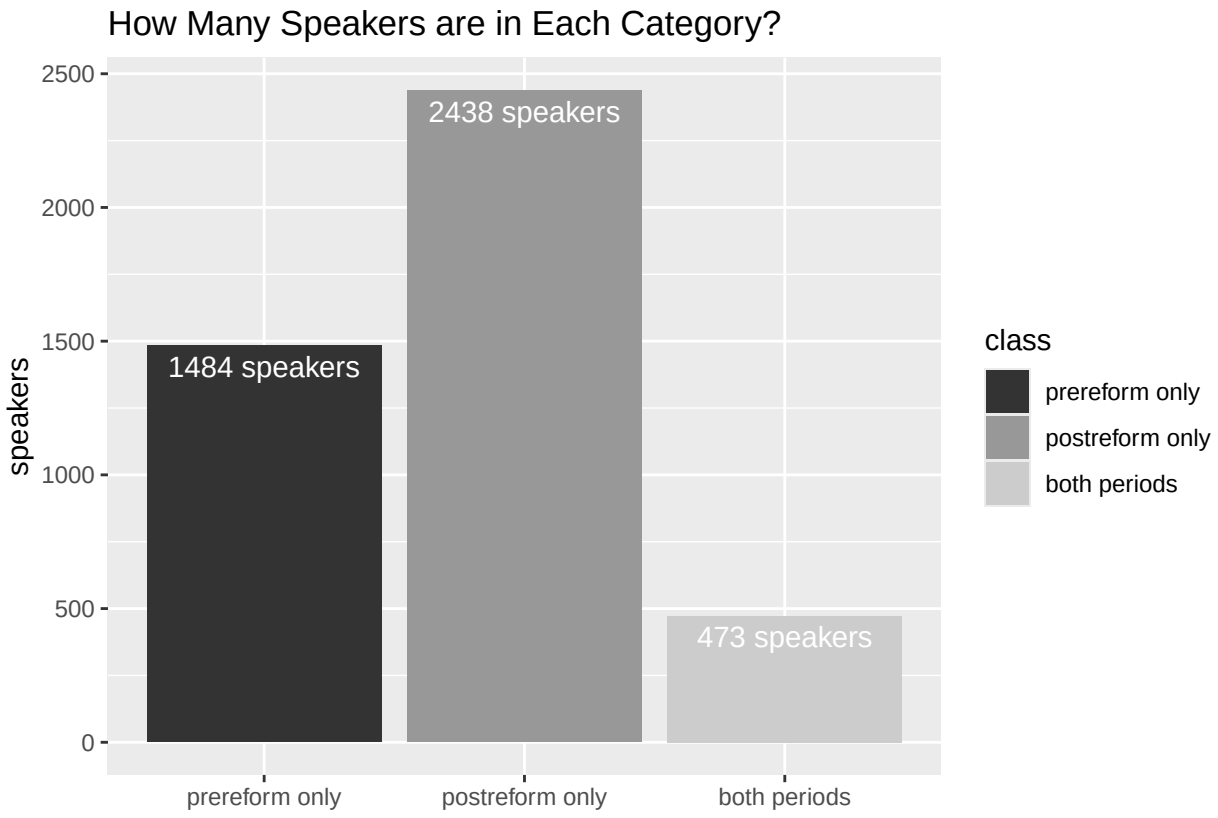
We might guess that it was vanishingly small, because a reform of historical significance could have swept away all incumbents. On the other hand, many historians of the 1832 Reform Bill have pointed out that it didn't shift the election process so drastically as had once been assumed.

In any case, to identify the speakers who appear in both the prereform and postreform datasets, we will use another function – `inner_join()` – to identify what is known as the "intersection" of the two datasets. That is, we are looking for speakers who occupy a row in both the `prereform_speakers_wordcount` data frame and the `postreform_speakers_wordcount` data frame. The syntax for `inner_join()` is just like that of `anti_join()`, but the result is to keep rows that appear in both datasets, not to exclude those rows.

```
## Joining with `by = join_by(suggested_speaker)`
```

```
## [1] 473
```

We now have actual numbers for the members of parliament recorded as only having spoken before the reform, those who were recorded as only having spoken after the reform, and those who were recorded as having spoken both before and after the reform. Let's represent our results visually.



Certain caveats are important for interpreting this number. While the speaker data gives us reliable insights into relative trends in membership and speech, the numbers are only so good as our dataset.

One of the great advantages of text mining is that we can analyze not just the speakers in parliament, but the consequences of the 1832 Reform Act for what was spoken about in parliament. Let's begin by asking what parliament spoke about before and after the reform.

0.3 Working With Word Count

```
## # A tibble: 7,503 x 5
##    suggested_speaker        year wordcount period    class
##    <chr>                   <dbl>     <int> <chr>     <chr>
##  1 abraham_hume_1561        1826       702 prereform prereform only
##  2 adolphus_dalrymple_1968  1831       457 prereform both periods
##  3 adolphus_dalrymple_1968  1832      1699 prereform both periods
##  4 alexander_baring_1475    1830      5671 prereform prereform only
##  5 alexander_baring_1475    1831      7135 prereform prereform only
##  6 alexander_dawson_2455    1826      1935 prereform prereform only
##  7 alexander_dawson_2455    1827       180 prereform prereform only
##  8 alexander_dawson_2455    1828      1580 prereform prereform only
```

```
##  9 alexander_dawson_2455    1830      1116 prereform prereform only
## 10 alexander_dawson_2455    1831       796 prereform prereform only
## # i 7,493 more rows
```

Note that we have created two columns that annotate the speeches and speakers in different ways. The "period" column refers to the year in question – were the n words said in 1831 prereform or postreform? The "class" column sorts the speakers themselves – did this speaker only speak in the prereform years, the postreform years, or in both periods?

We also have a wordcount column that reflects the words each speaker spoke per year.

Next, let's sort the data to find the speakers who spoke the most per year within each category – prereform, postreform, and continuously through the periods.

To find the speakers who spoke the most, we'll want to first create a new column, `wordcount_per_speaker`, which adds together each speaker's total wordcount. If we have multiple "wordcount" columns, we should add information about what each refers to. To make it less confusing, we'll rename our previous wordcount column `wordcount_per_year`. It's good practice to keep renaming fields in our datasets for clarity so that we don't get confused about what they mean. To calculate `wordcount_per_speaker`, we'll use the `group_by()` and `mutate()` functions that we've seen before. Note that we won't use `summarize()` in this case because we still want to keep the `wordcount_per_year()` field upon which the new count is based.

we'll use `group_by()` to group the data by "class," then use the `arrange()` and `slice_head()` functions to take the top wordcounts for each category.

```
## `summarise()` has grouped output by 'class'. You can override using the
## `.groups` argument.
```

```
## # A tibble: 15 x 4
## # Groups:   class [3]
##    class           suggested_speaker      wordcount_per_speaker top
##    <chr>           <chr>                                  <int> <lgl>
##  1 both periods    robert_peel_1664                     1982866 TRUE
##  2 both periods    henry_brougham_1679                  1616327 TRUE
##  3 both periods    joseph_hume_1712                     1336979 TRUE
##  4 both periods    john_russell_1885                    1046022 TRUE
##  5 both periods    daniel_oconnell_2552                  811831 TRUE
##  6 postreform only john_roebuck_2850                     241977 TRUE
##  7 postreform only frederick_shaw_2644                   211763 TRUE
##  8 postreform only thomas_wakley_3333                    205952 TRUE
##  9 postreform only george_howard_2466                    198439 TRUE
## 10 postreform only henry_ward_3175                       181587 TRUE
## 11 prereform only  william_huskisson_1230                252160 TRUE
## 12 prereform only  george_canning_1114                   222297 TRUE
## 13 prereform only  james_mackintosh_1886                 154994 TRUE
## 14 prereform only  henry_herbert_734                      66053 TRUE
## 15 prereform only  george_tierney_1068                    49551 TRUE
```

0.4 Annotating Our Data

Before we go any further, let's do some work to make the speaker names more elegant.

As the analyst will recall, because of the structure of our data, names that were algorithmically cleaned are labeled with a standardized name and number, e.g. "robert_peel_1665."

When we make a graph to share, we'd like to substitute Robert Peel's name for this awkward string. Therefore, we need to create a "key" where we can match the individual id "robert_peel_1665" to the name "Sir Robert Peel."

The names as given in the text are in the Hansard dataset under the `speaker` column. We can grab any one of them – minus the titles like "Lord Chancellor" with which some speakers are referenced – and it will read like a normal name. The `str_detect()` function is useful for searching for a string. `!str_detect()` will allow us to throw out names that include titles we don't want to use. The `sample_n()` function is useful here as it tells the computer to select a random pick from a set; in other words, if we've grouped Robert Peel's many possible names, and we have the set "Robert Peel," "Sir Robert Peel," and "Mr. Robert Peel," the computer is instructed to pull out one of the names at random.

After we choose one name to use from the full Hansard list of literal speaker references, we can use these names to annotate the previous dataset, `all_speakers_annotated`, which has `wordcount` (per speaker per year) and `speaker`.

Let's try it.

```
## Joining with `by = join_by(sentence_id)`
## Joining with `by = join_by(sentence_id)`
## Joining with `by = join_by(sentence_id)`
## Joining with `by = join_by(sentence_id)`
## Joining with `by = join_by(sentence_id)`
## Joining with `by = join_by(sentence_id)`
```

```
## # A tibble: 8 x 1
## # Groups:   suggested_speaker [8]
##   suggested_speaker
##   <chr>
## 1 edmund_peel_2792
## 2 frederick_peel_4084
## 3 jonathan_peel_2471
## 4 robert_peel_1664
## 5 robert_peel_1664|robert_peel_4101
## 6 robert_peel_1664|robert_peel_4101|john_neeld_3314
## 7 robert_peel_1664|robert_peel_4101|john_reid_2642
## 8 william_peel_1975
```

The results may not be impressive for the vast majority of speakers who spoke very little. But for major speakers like Robert Peel, the `speaker_key` dataset will help us reconcile the many names with which they appeared in the text with one numeric id number, then reconciling that numeric id with one recognizable name from the Hansard transcription.

1 Using Left Join to Annotate the Data

We've previously used set theory and "joins" to find the intersection and difference between two datasets. Here, we'll use another kind of join – the "union" – to annotate one dataset with information from another dataset.

That is, in `all_speakers_annotated`, we have a list of speakers and how many words they spoke every year. If we use `left_join()` to merge that dataset with the dataset `top_speakers()`, we'll have a record of whether each speaker was one of the most prolific speakers in their category. If we use `left_join()` to merge the dataset again with the dataset `speakers_key()`, we'll be able to use the literal transcriptions of speaker names to annotate the visualization later.

The function `left_join()` takes as its first argument the name of the dataset with which we're merging our data. A second optional argument is the "by" argument, which allows us to specify a column or set of columns to use for matching. In the case of the first join, we'll specify two fields, concatenated into a list, using the syntax `by = c("suggested_speaker", "class")`.

```
## Joining with `by = join_by(suggested_speaker)`
```

suggested_speaker	top	year	wordcount_per_speaker_per_year	class
edmund_peel_2792	NA	1831	2654	both periods
edmund_peel_2792	NA	1832	23	both periods
jonathan_peel_2471	NA	1831	34	prereform only
robert_peel_1664	TRUE	1823	27786	both periods
robert_peel_1664	TRUE	1824	46554	both periods
robert_peel_1664	TRUE	1825	35058	both periods

2 Visualize the Results

Sometimes the most effective way of analyzing data is to "inspect" the results of counting, as we have done henceforth. For simple results, like a list of names or words, we prefer a table.

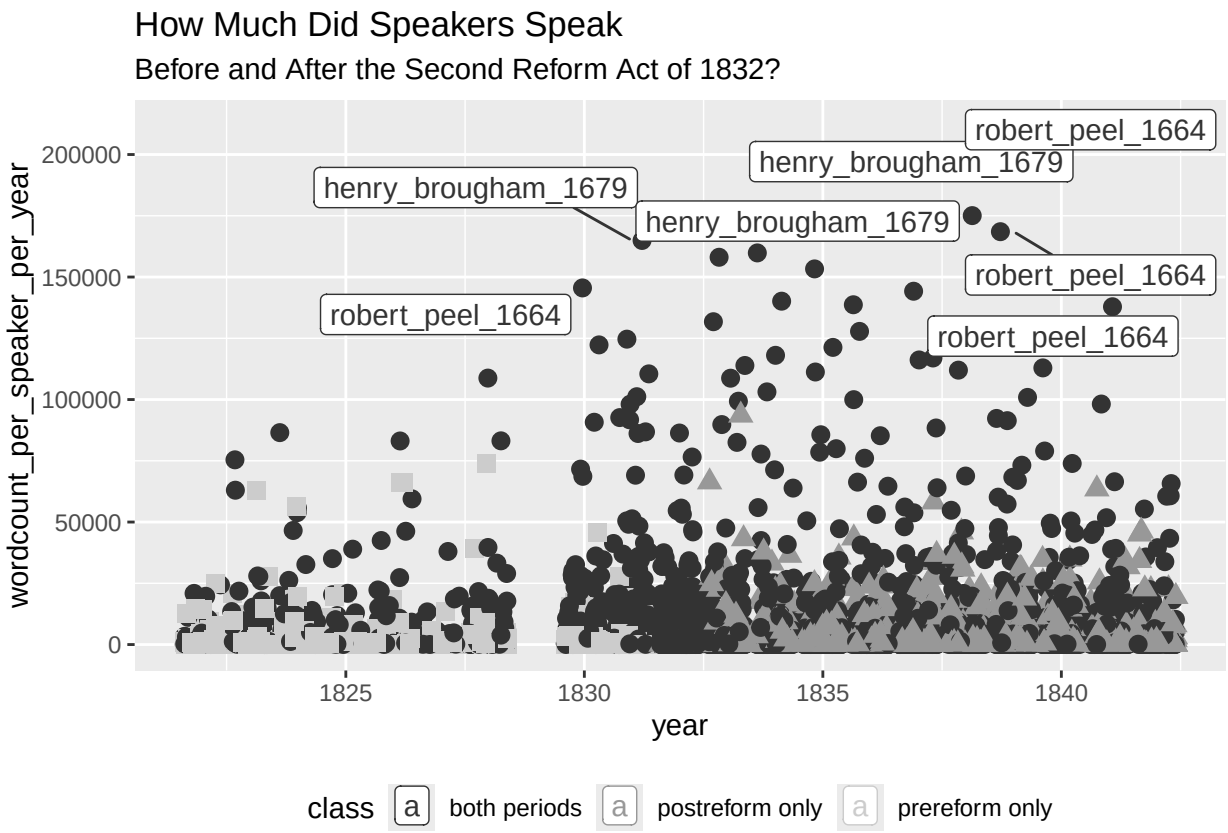
But for questions about information when several dimensions of data are related to each other – for instance speaker words over time when speakers have been categorized in three "classes" – visualization may be useful for conveying how many factors are related to each other. Next, we'll use `ggplot()` to create a visualization of the results as a timeline.

In the case of the data about speakers before and after 1832, we have rich information that deserves to be represented in an interesting way. We have three categories of speakers – prereform, postreform, and continuous speakers – and information about how many speakers are in each category. We also have information about how much each speaker spoke each year. Can we model the information together and learn something from the result?

in the code block below, we'll create a "dot plot," mapping one point for each of the 10,810 speakers in the data frame. We will use the color and shape of the dots to map the three classes of speakers – prereform, postreform, and continuous.

2.0.1 Contingent labeling

Because labeling all the speakers would result in an unreadable graphic, we will only label those speakers who are classified as `TRUE` in the column `top`, where we annotated if a speaker was among the top n speakers of each class by wordcount. The syntax for this operation is given inside the parentheses of `geom_label_repel()`, where the coder has an opportunity to point to a new dataset as the source of the text in the labels. Here, we tell the computer which dots to label and which to leave blank with the use of `subset(graph_ready, top == TRUE)`. Operations of this type are extremely useful for the kind of dense, text-forward information that implementors of text mining often want from their analysis.



What do we learn from this plot? A great deal. One is about the relatively higher counts of speeches in the 1830s than the 1820s, irrespective of the reform Act. Speeches by Lord Althorp, Robert Peel, and Henry Brougham regularly reached into range of 150,000 to 200,000 words – which if delivered at breakneck speed would have taken a stunning fifteen hours to deliver.

Even the speeches of lesser speakers were longer in the new decade, stretching towards 50,000 words instead of 20,000 words as the upper limit for most rank-and-file speakers – meaning that after the reform, multiple speakers each year addressed the house for five hours or more at a time.

We also learn that by and large it was the continuous speakers – not the prereform-only aristocratic appointees or the postreform-only members elected by the middle class – who spoke at the greatest length. Represented with black dots, these men included prime ministers like Robert Peel.

We can also use this visualization to note the few prereform-only speakers who delivered relatively long speeches, for instance Mr. Canning and Mr. Huskisson, the latter important to the Board of Trade, represented by light gray squares.

Equally exceptional were the few members of parliament elected only after 1832 who spoke with prominence, among them John Roebuck and Frederick Shaw. Both men whose careers were endorsed by reform swam in a sea of seasoned representatives who had first joined parliament in an earlier era.

Note that our visualization is only as good as the quality of the data. Where speakers have not been reconciled correctly – as is the case with "Lord Brougham" (classified as postreform only), who in reality is the same as "Henry Brougham" (classified as a continuous speaker) – they may appear as discrete speakers when indeed they are the same.

2.1 Using Distinctiveness to Investigate Language in the Period Before and After 1832

Interesting though it is to quantify the number of speakers and speeches before and after the Reform Act, far more consequential is the question of whether ideas or issues changed as new voters participated in the parliamentary process.

In general, historians have told us that after the 1832 Reform Act, Parliament began to pay more attention to issues that reflected the concerns of the growing middle class and urban population—concerns that had been largely ignored by the pre-reform, aristocracy-dominated Parliament. Key shifts included ending child labor, the repeal of the grain laws to make bread cheaper for urban workers, public health, tackling corruption, and grants for schools.

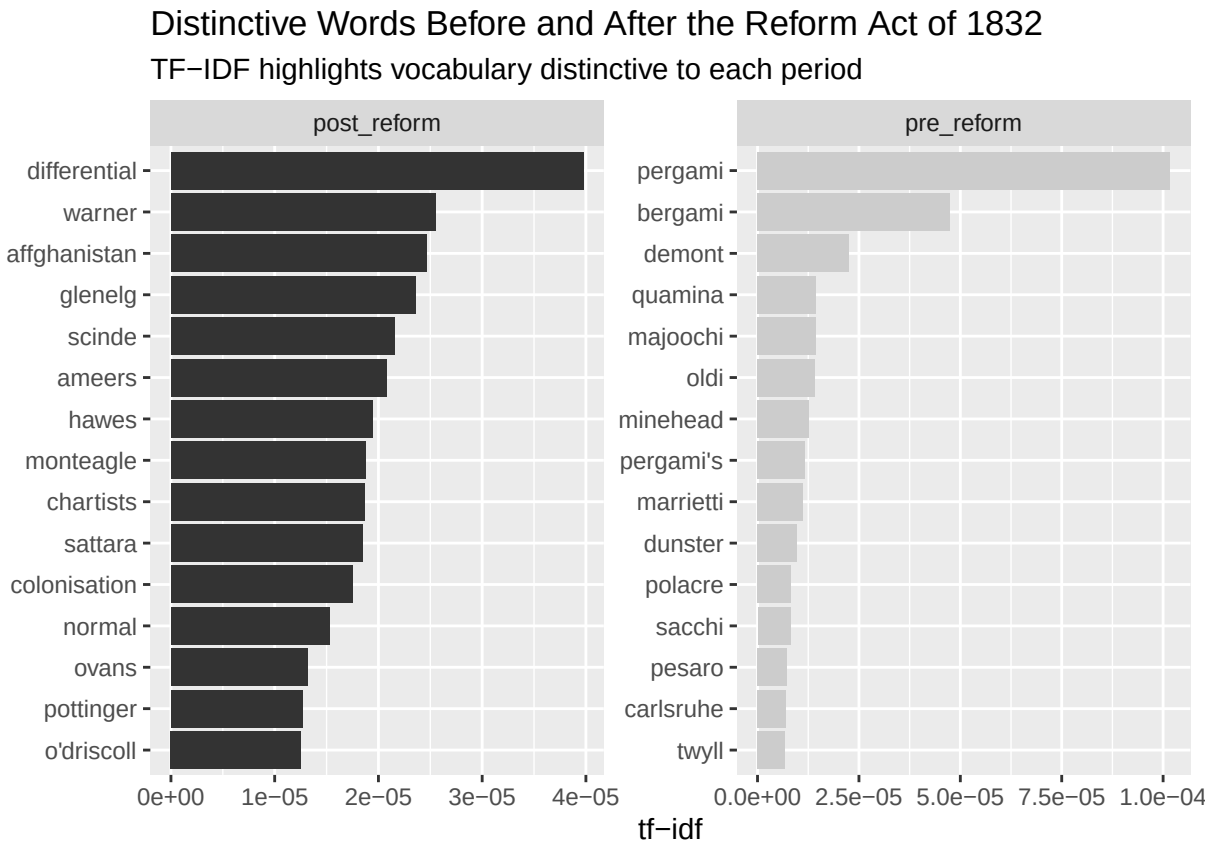
Yet the debates of parliament cover a diversity of issues far wider than the lexical set of changes that have been enshrined in history textbooks. If we really want to know, in full, how parliament sounded different before and after 1832, text mining provides one of our greatest possible tools for understanding.

Text mining gives us an opportunity to approach the issue on the basis of the words that speakers used. The tools of analyzing distinctiveness, introduced in the previous chapter, allow us to prioritize words that were used markedly more in one case or another. For example, we might become interested in which words were used the most before and after 1832.

By using tf-idf as our tool for understanding, we will privilege a strict filter: we will look for words spoken in 1822-32 that were almost never spoken in 1832-42, and vice-versa. What might the silences and expansions of the period reveal that we did not previously know?

```r
library(dplyr)
library(tidytext)
library(stringr)
library(lubridate)
library(ggplot2)

# Combine datasets
hansard <- bind_rows(
  annotated_1820,
  annotated_1830,
  annotated_1840
```

```r
)

# Define reform cutoff
reform_cutoff <- as.Date("1832-06-07")

hansard <- hansard %>%
  mutate(
    speechdate = as.Date(speechdate),
    period = if_else(speechdate < reform_cutoff, "pre_reform", "post_reform")
  )

# Tokenize + clean
tidy_words <- hansard %>%
  filter(!is.na(suggested_speaker)) %>%
  unnest_tokens(word, text) %>%
  anti_join(stop_words, by = "word") %>%
  filter(!str_detect(word, "^[0-9]+$"))

# Compute TF-IDF where document = period
period_tfidf <- tidy_words %>%
  count(period, word, name = "wordsperperiod") %>%
  bind_tf_idf(term = word,
    document = period,
    n = wordsperperiod
  )

# Get top words per period
top_words <- period_tfidf %>%
  group_by(period) %>%
  arrange(desc(tf_idf)) %>%
  slice_head(n = 15) %>%
  ungroup() %>%
  mutate(word = reorder_within(word, tf_idf, period))

# Plot
ggplot(top_words, aes(x = word, y = tf_idf, fill = period)) +
  geom_col(show.legend = FALSE) +
  facet_wrap(~period, scales = "free") +
  coord_flip() +
  scale_x_reordered() +
  scale_fill_grey() +
  labs(
    title = "Distinctive Words Before and After the Reform Act of 1832",
    subtitle = "TF-IDF highlights vocabulary distinctive to each period",
    y = "tf-idf",
    x = ""
  )
```



The results are interesting. One of the top most ranked prereform word is "quamina," a word that references Quamina Gladstone, the leader of the 1823 slave revolt in Demerara led by an enslaved carpenter born in Ghana and sold into slavery with his mother in 1817. Gladstone was prohibited from taking time off of work to look after his ill wife, Peggy. After Peggy's death, Quamina's son Jack Gladstone organized a peaceful strike of thousands. In the aftermath of the revolt, Quamina was assassinated, his body displayed in chains beside the public road.

Quamina Gladstone's story was important to prereform antislavery movements in Britain; Quamina's revolt was used to argue the case that the conditions of slavery were so miserable that they merited abolition. But By 1832-33, when slavery was under debate, Quamina's case was forgotten. The slave revolts still spoken about in 1833 were those in Haiti and Jamaica, associated with the murder of white slavers. White fear of violence was used as a pretext for extending slavery under the guise of "apprenticeships."

Other words too point at important differences between the prereform and postreform periods. Postreform, the terms "chartists" and "socialist" point to the movement to give the vote to the working class and experiments with government welfare associated at the time with the factory owner and utopian theorist Robert Owen. The word "normal" suggests statistics, but in context, parliamentary speakers of the 1830s typically invoked the word to refer to the new idea of founding colleges specifically for teachers, patterned after the famous Ecole Normale Superior in Paris.

Postreform terms such as "Glenelg" and "Normanby" reference individuals who played a major role in the expansion of empire. Charles Grant, 1st Baron Glenelg, served as Chief Secretary for Ireland after 1819 and

President of the Board of Trade after 1828. He helped to author the new Government of India Act of 1833, which transfered power away from the East India Company.

A more complete research project would use this list of distinctive words to investigate further; elsewhere, we discuss how to navigate the debates.

3 Exercises

1) This chapter builds on previous chapters, expanding what can be demonstrated with the tools provided so far. A great deal of space has been spent on creating elegant and informative visualizations, for example, timelines labeled with names that accurately reflect how an individual was described in the text, even while metadata behind the scenes ensures that we count one person per one dot. Why is it important to reference the names found in the original text? Why is it important to use metadata to ensure that one person gets one dot?

2) This chapter leverages a number of applications of distinctiveness measures – like tf-idf and log likelihood – to ask, in different ways, what is special about the speakers before and after the Reform Act? In your own words, explain what was done with the data in each of these exercises:

a) tf-idf by time period
b) comparing the most prolific speakers who were exclusively prereform against the most prolific speakers who were exclusively postreform

3) The approach profiled here generated analyses filled with obscure names, in part because we have used the stricter tf-idf rather than the more inclusive log likelihood measure. For the analyses of prereform and post reform speakers and their words, change the code so that the analysis uses log likelihood. Are the results any easier to interpret?

4) We have generated an analysis of 1832 and of 1867. Choose your favorite of the visualizations, and consult the version of this analysis for each event. What is the same in both events? What is different?

5) We have not supplied an interpretation for all of the obscure place names that occur in the comparison of speakers elected in one period versus those elected continuously. Choose one of the charts, and use your skills to navigate back to in-text mentions of these words. Based on what you have found, give an interpretation of the significance of distinctiveness measures for our understanding of the Reform Act.

6) In an essay of a page, explain why it is important to compare the results of different analyses rather than to choose one of these approaches at random. What do we get out comparing the results of different methods of analysis? Do you get more insight out of comparing different events (1832 vs 1867), or different aspects of the same event?